

Elixir

Управление программой и данными

Курс «Новые языки программирования»

Назарова Анастасия, Шилов Павел

СПбПУ, ВШПМиВФ

2026



```
result =  
  case File.read("data.txt") do  
    {:ok, text} ->  
      String.trim(text)  
    {:error, reason} ->  
      IO.puts("Ошибка: #{reason}")  
      nil  
  end
```

От выражения к потоку выполнения

В Elixir управление программой — это преобразование данных

 λ

Почти всё в Elixir является **выражением**

Конструкции управления возвращают значение.

 $\rightarrow$

Поток выполнения определяется **данными**

Ветвление происходит по значению и структуре данных (pattern matching).

Основной подход — **преобразование значений**

Императивный стиль с изменяемым состоянием не является основным.



Пример

```
age = 20

result =
  if age >= 18 do
    :adult
  else
    :minor
  end

result
# => :adult
```



В Elixir выражение всегда возвращает значение, поэтому его можно присваивать, передавать дальше и использовать в других выражениях.

Условия: **if**, **unless** и **cond**

Базовые конструкции выбора в Elixir



if

Проверяет условие и выбирает одну из двух веток.

Используется для простых условий.



unless

Обратная форма if: выполняется, если условие ложно.

Удобно, когда важен случай «не ...».



cond

Последовательно проверяет несколько условий и возвращает первую подходящую ветку.

Для нескольких вариантов выбора.

условие

выражение, которое проверяем



ветка

выбор нужной ветки выполнения



значение

результат выражения



В Elixir и if, и unless, и cond — это **выражения**. Они возвращают значение.



Пример

```
score = 82

result =
  cond do
    score >= 90 -> :excellent
    score >= 70 -> :good
    true -> :retry
  end

result
# => :good
```



Falsy значения в Elixir: только **false** и **nil**.

case и pattern matching

Ветвление по значению и структуре данных

Конструкция **case** позволяет выбирать ветку выполнения в зависимости от структуры данных.



Сопоставление структуры

Можно проверять не только значение, но и форму данных.



Несколько вариантов

Каждая ветка описывает отдельный шаблон.



Возвращает значение

case — это выражение, результат можно присваивать.



Пример

```
case File.read("data.txt") do
  {:ok, text} ->
    IO.puts("Файл прочитан")
    String.trim(text)

  {:error, reason} ->
    IO.puts("Ошибка: #{reason}")
    nil

  ->
    IO.puts("Неожиданный результат")
    nil
end
```



Pattern matching в case сравнивает значение с шаблоном слева. Первая подходящая ветка выполняется.

Guards — дополнительные условия

Расширяют pattern matching и позволяют более точно выбирать ветку

Что такое guards?

Guards — это дополнительные условия, которые проверяются вместе с pattern matching. Они позволяют выбирать нужную ветку в зависимости от свойств данных (тип, значение и т. д.).

Где используются?

-  в определениях функций
-  в case
-  в cond
-  в with (редко)

Пример

```
def classify(x) when x > 0, do: :positive
def classify(0), do: :zero
def classify(x) when x < 0, do: :negative
```

Пример

```
def check(value) do
  case value do
    x when is_integer(x) and x > 0 -> :positive
    0 -> :zero
    x when is_integer(x) and x < 0 -> :negative
    _ -> :unknown
  end
end
```

Вызов функции
classify(x)

Pattern matching

+

Guard (доп. условие)

Выбор ветки

Допустимые выражения в guards

- ✓ проверка типа (is_integer/1 и др.)
- ✓ сравнения (>, <, == и др.)
- ✓ логические операторы (and, or, not)
- ✓ числовые условия
- ✓ несколько guards через and
- ✓ другие чистые и безопасные функции

Итерация без классических циклов

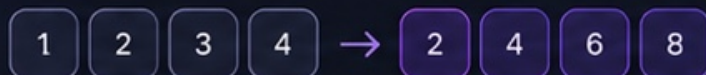
В Elixir обычно обрабатывают коллекции, преобразуя их, а не изменяя счётчик

Enum.map — преобразование

Применяет функцию к каждому элементу и возвращает новый список.

Пример

```
Enum.map([1, 2, 3, 4], fn x -> x * 2 end)
# => [2, 4, 6, 8]
```

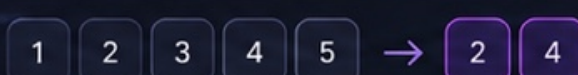


Enum.filter — фильтрация

Оставляет только те элементы, для которых функция возвращает true.

Пример

```
Enum.filter([1, 2, 3, 4, 5], fn x ->
  rem(x, 2) == 0
end)
# => [2, 4]
```

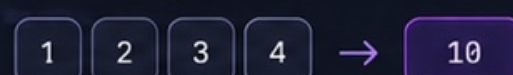


Enum.reduce — свёртка

Постепенно объединяет элементы в одно значение.

Пример

```
Enum.reduce([1, 2, 3, 4], 0, fn x, acc ->
  acc + x
end)
# => 10
```



Comprehension — генерация и фильтрация

Удобный способ создавать новые коллекции с условиями.

Пример

```
for x <- 1..5, x * x < 20, do: x * x
# => [1, 4, 9, 16]
```



Comprehension похож на математическое описание множества: «возьми x из диапазона, отфильтруй по условию и верни выражение».

Enum — работа с перечислимыми данными

Модуль Enum работает с любыми структурами, которые реализуют протокол Enumerable (списки, диапазоны, карты и др.).

Пример

```
Enum.map(1..3, &(&1 + 1)) # => [2, 3, 4]
Enum.to_list(1..5)        # => [1, 2, 3, 4, 5]
Enum.sum([1, 2, 3, 4])    # => 10
```

with — цепочки успешных операций

Позволяет последовательно выполнять операции и обрабатывать ошибки

Основная идея

with выполняет цепочку выражений, каждое из которых должно сопоставиться с заданным шаблоном.

Если любое выражение не совпадает с шаблоном, выполняется блок else.

Когда использовать?

- ✓ последовательные операции
- ✓ работа с `{:ok, ...}` / `{:error, ...}`
- ✓ чтение файлов, запросы, парсинг
- ✓ избегание вложенных case
- ✓ более линейный и читаемый код

Пример

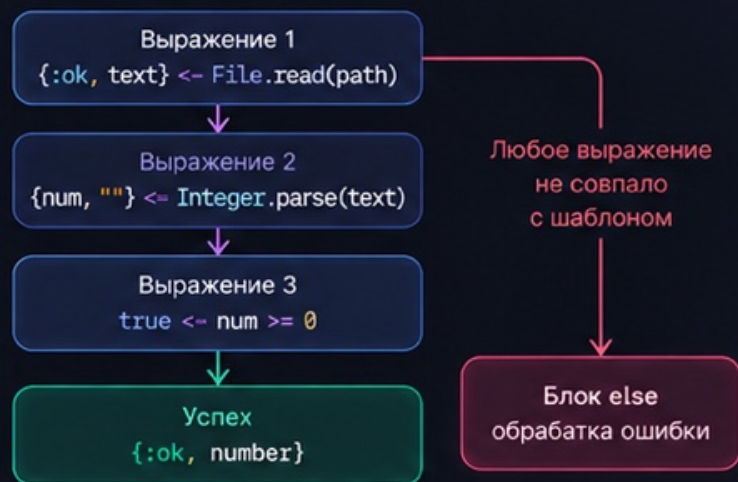
```
def parse_number_from_file(path) do
  with {:ok, text} <= File.read(path),
       {number, ""} <= Integer.parse(String.trim(text)),
       true <= number >= 0 do
    {:ok, number}
  else
    {:error, reason} ->
      {:error, {:read_error, reason}}
    :error ->
      {:error, :parse_error}
    false ->
      {:error, :negative_number}
  end
end
```

Вызовы:

```
iex> parse_number_from_file("data.txt")
{:ok, 42}

iex> parse_number_from_file("bad.txt")
{:error, {:read_error, :enoent}}
```

Как это работает?



Преимущества

- ✓ убирает вложенные case
- ✓ делает код линейным и читаемым
- ✓ удобно для работы с результатами операций
- ✓ позволяет обрабатывать разные типы ошибок
- ✓ хорошо сочетается с pattern matching

Ошибки как данные

В Elixir обычные ошибки представляются значениями, а не исключениями

Стандартная конвенция

Большинство функций, которые могут завершиться с ошибкой, возвращают кортеж:



`{:ok, value}`

операция выполнена успешно



`{:error, reason}`

произошла ожидаемая ошибка

Пример: чтение файла

```
case File.read("data.txt") do
  {:ok, text} ->
    IO.puts("Файл прочитан")
    IO.puts(String.trim(text))

  {:error, reason} ->
    IO.puts("Ошибка: #{reason}")
end
```

`{:ok, "Hello world\n"}`
если файл существует

`{:error, :enoent}`
если файла нет

Две версии функции

Многие функции имеют две версии: безопасную (с возвратом кортежа) и версию с `!`, которая вызывает исключение при ошибке.

Безопасная версия

Elixir



`File.read("data.txt")`

`# => {:ok, text}`

`# => {:error, reason}`

Версия с `!`

Elixir



`File.read!("data.txt")`

`# => text`

`# => выбрасывает
исключение при ошибке`



Версия с `!` удобна, когда ошибка невозможна или должна привести к немедленному завершению программы (например, в скриптах или во время запуска).

Ожидаемые и неожиданные ошибки



Ожидаемая ситуация

Файл не найден, некорректные данные и т. д. → вернуть `{:error, reason}`



Неожиданная ситуация

Логическая ошибка в коде, нарушение инвариантов → вызвать исключение

Использование с pattern matching

Удобно сразу сопоставлять результат:

```
{:ok, text} = File.read("config.json")
# text содержит содержимое файла
```



Такой способ подходит, когда мы уверены, что операция успешно выполнится. При ошибке будет вызвано исключение `MatchError`.

Распространённые причины ошибок



`:enoent`

— файл или директория не найдены



`:eaccess`

— нет прав доступа



`:EINVAL`

— неверные аргументы



`:eexist`

— уже существует



`:ECONNREFUSED`

— соединение отклонено

Исключения и отладка

Исключения используются для действительно исключительных ситуаций

Когда использовать исключения?



Подходит

неожиданные ситуации, из которых нельзя корректно продолжить работу (внутренняя ошибка, неверное состояние)



Не подходит

обычные ошибки при работе с данными (чтение файлов, запросы и т. д.) — для этого лучше возвращать `{:error, reason}`

Пример: raise и try/rescue



Elixir

```
def divide(_a, 0) do
  raise ArgumentError, "Деление на ноль"
end

def divide(a, b), do: a / b

try do
  result = divide(10, 0)
  IO.puts("Результат: #{result}")
rescue
  e in ArgumentError ->
    IO.puts("Перехвачено исключение: #{e.message}")
end
```

Типичные исключения

ArgumentError	некорректные аргументы
FunctionClauseError	нет подходящего шаблона функции
MatchError	не удалось сопоставить значение
KeyError	ключ не найден в Map
File.Error	ошибки файловой системы



Исключения в Elixir не замещают `{:ok, value}` / `{:error, reason}`, а дополняют их.

try / rescue / else / after



Elixir

```
try do
  {:ok, text} = File.read("data.txt")
  process(text)
rescue
  e => IO.puts("Ошибка: #{inspect(e)}")
else
  result -> IO.puts("Успех: #{result}")
after
  IO.puts("Этот блок выполняется всегда")
end
```

try

защищённый блок кода

rescue

обработка исключений

else

выполняется, если исключения не было

after

выполняется всегда (например, закрыть файл)

Отладка в Elixir



Elixir

```
value
|> transform()
|> dbg(label: "После transform")
|> IO.inspect(pretty: true)
```

**dbg/2**

показывает значение и место вызова в коде

**IO.inspect/2**

удобный вывод структур

**stacktrace**

подробная информация об исключении



Для отладки удобно комбинировать `dbg/2`, `IO.inspect/2` и понятные сообщения в исключениях.

ExUnit: встроенное тестирование

Простой и мощный инструмент для надёжного кода

Основные понятия

- 1 ExUnit.Case**
Базовый модуль для тестов. Подключает нужные импорты и настройки.
- 2 test**
Определяет отдельный тест.
- 3 assert / refute**
Проверяют, что выражение истинно или ложно.
- 4 doctest**
Автоматически проверяет примеры из документации.

Пример: тест модуля

```
Elixir
defmodule MyApp.MathTest do
  use ExUnit.Case, async: true

  test "сложение чисел" do
    assert MyApp.Math.add(2, 3) == 5
  end

  test "некорректный аргумент" do
    assert_raise ArgumentError, fn ->
      MyApp.Math.add(2, "3")
    end
  end

  doctest MyApp.Math
end
```

Запуск тестов

```
Elixir
$ mix test

Compiling 1 file (.ex)
Generated my_app app
..
Finished in 0.03 seconds (0.00s async, 0.03s sync)
3 tests, 0 failures
```

Полезные возможности



async: true
Параллельный
запуск тестов



assert_raise
Проверка ошибок
и исключений



doctest
Тесты примеров
из документации



capture_log
Проверка логов
во время тестов

Пример: doctest

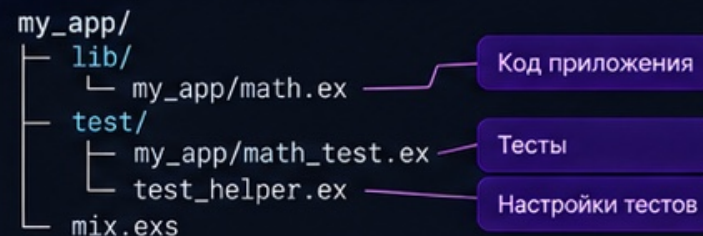
```
Elixir
defmodule MyApp.Math do
  @doc """
  Складывает два числа.

  iex> MyApp.Math.add(2, 3)
  """ 5
  def add(a, b), do: a + b
end
```

Пример: тест с захватом логов

```
Elixir
test "пишет лог" do
  log = capture_log(fn ->
    require Logger
    Logger.info("тестовое сообщение")
  end)
  assert log =~ "тестовое сообщение"
end
```

Структура тестового проекта



Модули и пространства имён

Организация кода, разделение логики и управление видимостью функций

Что такое модуль?

Модуль (module) — это основной способ организации кода в Elixir. Модули объединяют функции, позволяют разделять код на логические части и использовать пространства имён.



Структурирование кода по функциональности



Избежание конфликтов имён



Переиспользование и поддержка



Удобное тестирование отдельных модулей

Пример: простой модуль

```
Elixir
defmodule MyApp.Users do

  def find(id) do                # публичная функция
    # поиск пользователя по id
    {:ok, %{id: id, name: "User"}}
  end

  def list do                    # ещё одна публичная функция
    # список пользователей
    [:user1, :user2]
  end

  defp format_user(user) do      # приватная функция
    "#{user.id}: #{user.name}"
  end
end
```

Пространства имён и вложенные модули

```
Elixir
defmodule MyApp do
  defmodule Accounts do
    def get_account(id), do: ...
  end

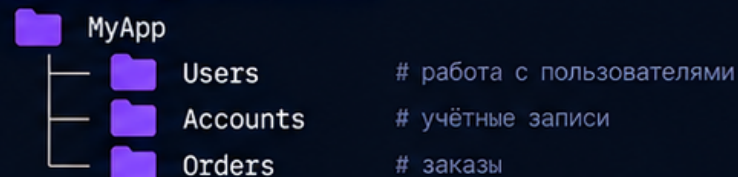
  defmodule Accounts.User do
    def struct, do: %{id: nil, name: nil}
  end
end
```

MyApp — корневой модуль

MyApp.Accounts — вложенный модуль

MyApp.Accounts.User — вложенный модуль внутри модуля

Дерево модулей проекта



Публичные и приватные функции



def

Публичная функция.
Доступна из других модулей.

```
def get(id), do: ...
```



defp

Приватная функция.
Доступна только внутри модуля.

```
defp helper(), do: ...
```

Вызов функций из других модулей

```
Elixir
alias MyApp.Users
Users.find(1)           # {:ok, %{...}}
MyApp.Users.list()      # [:user1, :user2]

# приватную функцию вызвать нельзя:
# MyApp.Users.format_user(user) # ошибка
```

Ключевые особенности



Имена модулей используют CamelCase: MyApp.Users, MyApp.Accounts



Вложенные модули позволяют строить иерархию (MyApp.Accounts.User)



Публичные функции вызываются из других модулей, приватные — только внутри



Модули — основа для больших приложений и удобной структуры проекта

alias, import, require и use

Удобный доступ к модулям и подключение их функциональности

Для чего используются?

**alias**

создаёт короткое имя для модуля

**import**

импортирует функции модуля в текущую область

**require**

даёт доступ к макросам модуля

**use**

подключает функциональность модуля (часто через макросы)

Пример использования



Elixir

```
defmodule MyApp do
  alias MyApp.Users      # короткое имя модуля
  import Enum             # импорт функций
  require Logger          # доступ к макросам
  use GenServer           # подключение поведения

  def example do
    users = Users.list()  # вместо MyApp.Users.list()
    doubled = map([1, 2, 3], &(&1 * 2)) # map из Enum
    Logger.info("Пользователей: #{length(users)}")
    {:ok, doubled}
  end
end
```

Как это работает?

alias Module

Создаёт сокращённое имя. Упрощает обращение к модулю.

import Module

Импортирует публичные функции модуля, можно использовать без указания имени модуля.

require ModuleНужен для использования макросов модуля (например, `Logger`, по условию в компиляции).**use Module**Вызывает макрос `__using__/1`, подключает функциональность модуля (например, `GenServer`, `Supervisor`, `Phoenix` и др.).

Пример без alias и с alias

Без alias



Elixir

```
def get_user(id) do
  MyApp.Users.get(id)
end

def create_user(params) do
  MyApp.Users.create(params)
end
```

С alias



Elixir

```
alias MyApp.Users

def get_user(id) do
  Users.get(id)
end

def create_user(params) do
  Users.create(params)
end
```

Import: функции доступны напрямую



Elixir

```
import Enum

def process(list) do
  list
  |> map(&(&1 * 2)) # без Enum.
  |> filter(&(&1 > 10))
end
```

Require и use: макросы и поведения



Elixir

```
require Logger

def log_info(msg) do
  Logger.info(msg) # макрос (работает в компиляции)
end

use GenServer # подключает поведение GenServer
# добавляет колбэки и вспомогательные функции
```

Коллекции и ввод-вывод

Основные структуры данных и работа с внешним миром

Основные коллекции в Elixir



List

Упорядоченная коллекция любых значений. Часто используется с Enum.

```
list = [1, 2, 3]
head = hd(list) # 1
tail = tl(list) # [2, 3]
```



Tuple

Фиксированный набор значений произвольных типов.

```
tuple = {1, "ok", :atom}
{a, b, c} = tuple
# pattern matching
```



Map

Коллекция пар "ключ → значение". Ключи — любые значения.

```
map = %{name: "Elixir",
         year: 2011}
map[:name] # "Elixir"
map.year # 2011
```



MapSet

Множество уникальных элементов (на основе Map).

```
set = MapSet.new([1, 2, 2])
MapSet.member?(set, 1) # true
MapSet.to_list(set) # [1, 2]
```



Keyword List

Список пар {ключ, значение} с атомами в качестве ключей. Удобен для передачи опций.

```
opts = [port: 4000, host: "localhost"]
Keyword.get(opts, :port) # 4000
Keyword.get(opts, :host) # "localhost"
```

Ввод и вывод. Работа с файлами



Elixir

```
name =
  IO.gets("Имя: ") # читаем строку из консоли
  |> String.trim() # убираем символ новой строки

File.write!("name.txt", name) # записываем в файл

{:ok, text} = File.read("name.txt") # читаем из файла
IO.puts(text) # выводим содержимое
```

Что здесь происходит?

- 1** `IO.gets/1` читает строку из консоли (в конце есть `\n`)
- 2** `String.trim/1` удаляет пробельные символы (в том числе `\n`)
- 3** `File.write!/2` записывает строку в файл (вариант с исключением)
- 4** `File.read/1` читает файл, возвращает `{:ok, текст}` или `{:error, причина}`
- 5** `IO.puts/1` выводит текст в консоль

Связь с другими темами

- |>** Оператор pipeline делает код последовательным и читаемым.
- 📄** I/O и работа с файлами позволяют взаимодействовать с внешним миром.
- 📁** Pattern matching используется для обработки результатов `{:ok, value}`.
- 🗄️** Коллекции (списки, карты и др.) удобны для хранения и обработки данных из файлов, ввода и т.д.

Protocols, behaviours и обобщённые типы

Полиморфизм, контракты и почему в Elixir нет классических generics

Protocol: полиморфизм по типу данных

Протокол задаёт общий интерфейс для разных типов данных. Позволяет реализовать одну и ту же функцию для разных типов.

defprotocol

```
defprotocol MyFormatter do
  def format(data)
end

defimpl MyFormatter, for: Integer do
  def format(data), do: "Число: #{data}"
end

defimpl MyFormatter, for: List do
  def format(data), do: "Список: #{Enum.join(data, ", ")}"
end
```

Примеры из стандартной библиотеки



String.Chars
преобразование
к строке (to_string/1)



Enumerable
обход коллекций
(Enum.map/2, Enum.each/2
и др.)

Behaviour: контракт для модулей

Behaviour задаёт набор функций, которые должен реализовать модуль. Используется для общей логики и замены реализаций.

defmodule

```
defmodule MyRepo do
  @callback get(id) :: {:ok, any()} | {:error, any()}
  @callback create(params) :: {:ok, any()} | {:error, any()}
end

defmodule MemoryRepo do
  @behaviour MyRepo

  def get(id), do: {:ok, %{id: id}}
  def create(params), do: {:ok, Map.put(params, :id, 1)}
end
```

Где используется?



Хранилища данных
(например, Ecto.Adapter)



Сервисы и плагины



Возможность подменять
реализации



Проверка на этапе
компиляции, что все
функции реализованы

Обобщённые типы в Elixir



Классических generic-типов типа `List<T>` в Elixir нет.

Elixir — функциональный и исторически динамический язык. Функции работают с различными структурами данных через pattern matching и guards. Общий интерфейс и полиморфизм реализуются через протоколы и поведения, а не через generics.

Почему нет generics?

- 1 Язык динамически типизирован, тип определяется во время выполнения.
- 2 Pattern matching позволяет удобно работать с любыми структурами данных без явного указания типов.
- 3 Полиморфизм достигается через protocols и behaviours.
- 4 Система типов ориентирована на практическую и выразительность, а не на параметрические типы.

Сравнение подходов

ООП / Java / C++

Интерфейс

Generic-тип

Работа с данными

Полиморфизм



Elixir

Behaviour / Protocol

Функции + pattern matching

Динамическая типизация

Протоколы и pattern matching

Что мы получили

Elixir-программа — это преобразование данных

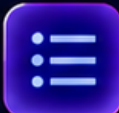
case / cond — выбор



Условная логика через pattern matching.

```
case result do
  {:ok, value} -> handle(value)
  {:error, reason} -> handle_error(reason)
end
```

Enum — обработка коллекций



Удобная работа со списками и другими коллекциями.

```
list
|> Enum.filter(&(&1 > 0))
|> Enum.map(&(&1 * 2))
```

with — последовательность операций



Удобная цепочка действий с обработкой ошибок.

```
with {:ok, data} <- read_file(),
     {:ok, parsed} <- parse(data),
     {:ok, result} <- process(parsed) do
  {:ok, result}
end
```

{:ok,} — ошибки как значения



Явное и удобное описание результата операций.

```
case File.read("file.txt") do
  {:ok, text} -> IO.puts(text)
  {:error, reason} -> IO.puts(reason)
end
```

**Elixir-программа =
преобразование
данных**

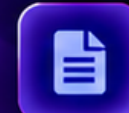
modules — организация кода



Разделение на модули и пространства имён.

```
defmodule MyApp.Users do
  def find(id), do: ...
end
```

IO / File — взаимодействие с внешним миром



Чтение ввода, вывод в консоль, работа с файлами.

```
name = IO.gets("Имя: ") |> String.trim()
File.write!("name.txt", name)
{:ok, text} = File.read("name.txt")
IO.puts(text)
```

protocols / behaviours — абстракции



Общий интерфейс для типов данных и модулей.

```
defprotocol MyFormatter do
  def format(data)
end
defmodule MyRepo do
  @behaviour MyRepoBehaviour
  def get(id), do: {:ok, %{id: id}}
end
```

коллекции — удобные структуры данных



Списки, кортежи, карты, множества, и списки ключей.

```
list = [1, 2, 3]
tuple = {:ok, 42}
map = %{name: "Elixir", year: 2026}
set = MapSet.new([1, 2, 2])
opts = [port: 4000, host: "localhost"]
```



Не изменяем состояние — преобразуем значения.

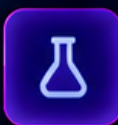
Литература и материалы

Полезные ресурсы для дальнейшего изучения



Elixir Documentation

Официальная документация языка и экосистемы.



ExUnit

Тестирование в Elixir. Основы и примеры.



Getting Started

Введение в Elixir. Установка, первые шаги, примеры.



Protocol

Протоколы и полиморфизм по типу данных.



Enum

Обработка коллекций. Основные функции и примеры.



Behaviour

Контракты для модулей. Создание и использование.



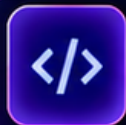
File / IO

Работа с файлами и вводом-выводом. Чтение, запись, обработка данных.



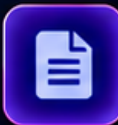
Mix

Инструмент сборки проектов. Зависимости, задачи, релизы.



Kernel.SpecialForms

Специальные формы языка (case, cond, if, with и др.).



HexDocs

Документация библиотек из экосистемы Elixir.



Дополнительно

- Официальные руководства по популярным библиотекам (Ecto, Phoenix и др.).
- Примеры проектов на GitHub.
- Сообщество Elixir (форумы, чаты, Discord).



Elixir

ПРОСТОТА • МОЩЬ • УДОВОЛЬСТВИЕ



Спасибо за внимание!

